

Competitive Programming

2D Prefix Sums & Difference Arrays

Basics

2D Prefix Sums - extends the 1D concept to matrices to query submatrix sums in $O(1)$.

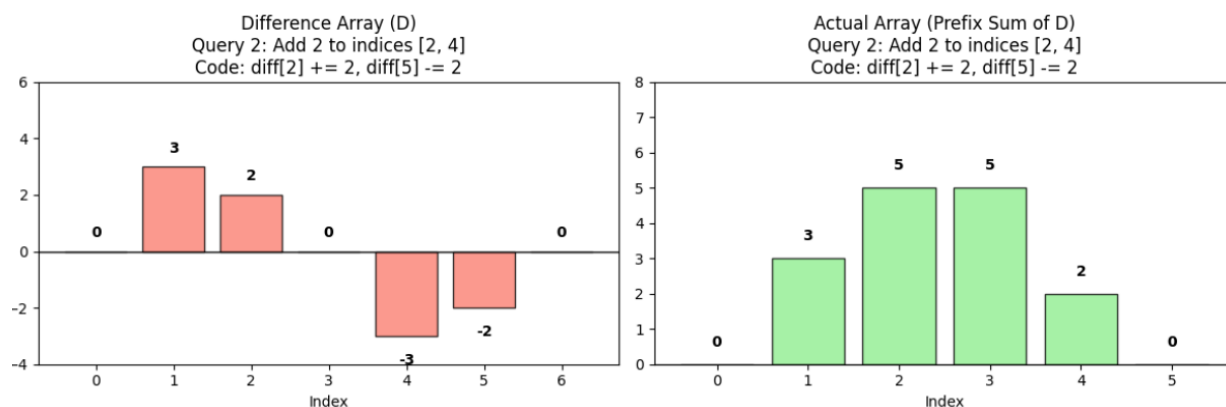
- Build: $P[r][c] = \text{matrix}[r][c] + P[r-1][c] + P[r][c-1] - P[r-1][c-1]$
- Query ($r1, c1$ to $r2, c2$): $\text{Sum} = P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1]$

i/j	0	1	2	3	4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19
4	20	21	22	23	24

i/j	0	1	2	3	4
0	0	1	3	6	10
1	5	12	21	32	45
2	15	33	54	78	105
3	30	64	102	144	190
4	50	105	165	230	300

Difference Arrays (1D Sweep Line) - used when you need to apply multiple additions/subtractions to a range $[L, R]$ and only need to know the final array at the end. Instead of a for loop, do $\text{diff}[L] += \text{val}$ and $\text{diff}[R + 1] -= \text{val}$.

After all queries, reconstruct the array by taking the prefix sum of the diff array: $A[i] = A[i-1] + \text{diff}[i]$.



Prefix XOR - XOR is its own inverse. Just like $\text{Sum}(i, j) = \text{Pref}[j] - \text{Pref}[i-1]$, for XOR: $\text{XOR}(i, j) = \text{PXOR}[j] \oplus \text{PXOR}[i-1]$

Problem Solving Patterns

1. **State-Compression Prefix Sums** - used when the problem asks for subarrays where elements satisfy even/odd counts.
Instead of summing, the prefix state is a bitmask. If 'a' is odd, flip the 0th bit. If the same bitmask appears at index i and index j, the subarray between them has an even count of everything (because odd - odd = even). Store Mask-> First Seen Index in a Hash Map.
2. **2D Difference Array** - to add V to a submatrix from [r1, c1] to [r2, c2], update 4 points:
 - `diff[r1][c1] += V`
 - `diff[r2+1][c1] -= V`
 - `diff[r1][c2+1] -= V`
 - `diff[r2+1][c2+1] += V`
3. **Prefix Sum + Modulo Arithmetic** - used to find subarrays divisible by K. Store the prefix sum modulo K in a Hash Map. If $P[j] \% K == P[i-1] \% K$, then the subarray sum $P[j] - P[i-1]$ is a multiple of K.

Contest Questions

1. Number of Sub-arrays With Odd Sum
2. Range Addition
3. Range Sum Query 2D – Immutable
4. Matrix Block Sum
5. Continuous Subarray Sum
6. Find the Longest Substring Containing Vowels in Even Counts
7. Stamping the Grid
8. Maximum XOR of Two Non-Overlapping Subtrees

Dynamic Programming - Digit DP

Basics

Digit DP is a pattern used to count the number of integers in a range [L, R] that satisfy a specific property. Because the range is too large to iterate ($O(N)$), we treat the numbers as strings and construct valid numbers digit by digit ($O(\log N)$).

Counting in [L, R] is almost always converted to: **Solve(R) - Solve(L - 1)**.

A standard Digit DP uses a top-down memoized function with 4 common parameters: **dp(i, tight, leadingZero, state)**

- i: the current digit position we are placing.

- If **tight == true**, the prefix we've built matches the upper bound R exactly. We can only pick digits from 0 up to R[i].
- If **tight == false**, the prefix is already strictly smaller than R. We are free to pick any digit from 0 to 9.
- **leadingZero**: true if we have only placed 0s so far. Crucial for problems where the length of the number matters or if 0 shouldn't be counted in a specific way.
- **state**: what we are keeping track of (sum of digits, counts of a specific number, modulo remainders, bitmask of used digits, etc.).

Problem Solving Pattern

1. **The Core Loop** - inside the recursive function, calculate the **limit = (tight ? R[i] - '0' : 9)**. Loop for (int d = 0; d <= limit; d++), make the choice, and recursively call the function.
2. **Updating Tight** - the next tight state becomes **tight && (d == limit)**.
3. **Memoization** - only memoize the state when **tight == false** (and sometimes leadingZero == false). If tight == true, the branch is bounded by the specific string R, which is a unique path and shouldn't be cached for general use.
4. **Modulo states** - if the problem asks for numbers divisible by K, pass **(current_rem * 10 + d) % K** into the next state.

Contest Questions

1. (**) Number of Digit One
2. (**) Numbers At Most N Given Digit Set
3. (**) Count of Integers
4. (**) Numbers With Repeated Digits
5. (**) Count Good Integers in a Range
6. (**) Count Special Integers

Binary Lifting

Basics

Binary Lifting is a DP technique used on trees (and sometimes graphs) to make jumps along paths efficiently. Instead of moving up a tree one parent at a time $O(n)$, we precompute jumps in powers of 2, allowing us to traverse paths in $O(\log N)$ time.

The core idea relies on the fact that any number k can be represented as a sum of powers of 2.

Create a 2D array **up[node][i]** which stores the 2^i -th ancestor of node.

- **Base Case**: **up[node][0] = parent[node]** (the $2^0 = 1$ st ancestor is just the parent).

- Recurrence: $up[node][i] = up[up[node][i-1]][i-1]$

Example: To find 4th ancestor, we find the 2nd ancestor of 2nd ancestor.

Problem Solving Patterns

1. **The K-th Ancestor**- used to find the k-th parent of a node efficiently. Iterate through the bits of k from 0 to logN. If the i-th bit of k is set, jump the node (**node = up[node][i]**).
2. **Lowest Common Ancestor (LCA)** - used to find the lowest shared parent of nodes u and v in $O(\log N)$ time for multiple queries.
 - o **Step 1:** find the depths of u and v. If u is deeper, use the k-th ancestor logic to jump u upwards until $depth[u] == depth[v]$.
 - o **Step 2:** if $u == v$, return u.
 - o **Step 3:** iterate i from logN down to 0. If jumping both nodes by 2^i lands them on different nodes ($up[u][i] != up[v][i]$), make the jump: $u = up[u][i]$, $v = up[v][i]$.
 - o **Step 4:** Return the immediate parent $up[u][0]$.

Contest Questions

1. Kth Ancestor of a Tree Node
2. Lowest Common Ancestor of a Deepest Leaves
3. [CSES] Company Queries I
4. [CSES] Company Queries II